P2906R0
Structured bindings for `std::extents`

Published Proposal, 2023-05-29

This version:
http://wg21.link/P2906

Author:
Bernhard Manfred Gruber (CERN)

Audience:
LEWG

Project:
ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Source:
GitHub

Abstract

This paper proposes to add support for structured bindings to `std::extents`.

Table of Contents

1 Motivation and Scope

2 Impact On the Standard

3 Design Decisions

3.1 Handling static extents

3.2 Modification of extents

4 Implementation Option A: demote static extents to runtime values

5 Implementation Option B: retaining static extents

6 Polls

7 Wording

7.1 Feature-test macro

8 Acknowledgements

References

Informative References

§ 1. Motivation and Scope

[P0009r18] proposed `std::mdspan`, which was approved for C++23. It comes with the utility class template `std::extents` to describe the integral extents of a multidimensional index space. Practically, `std::extents` models an array of integrals, where some of the values can be specified at compile-time. However, `std::extents` behaves very little like an array. A notable missing feature are structured bindings, which would come in handy if the extents of the individual dimensions need to be extracted:

Before

```
std::mdspan<double,
    std::extents<I, I1, I2, I3>, L, A> mdspan;
const auto& e = mdspan.extents();
for (I z = 0; z < e.extent(2); z++)
    for (I y = 0; y < e.extent(1); y++)
        for (I x = 0; x < e.extent(0); x++)
            mdspan[z, y, x] = 42.0;

const auto total =
    e.extent(0) * e.extent(1) * e.extent(2);
```

After

```
std::mdspan<double,
    std::extents<I, I1, I2, I3>, L, A> mdspan;
const auto& [depth, height, width] = mdspan.exten
for (I z = 0; z < depth; z++)
    for (I y = 0; y < height; y++)
        for (I x = 0; x < width; x++)
            mdspan[z, y, x] = 42.0;

const auto total =
    width * height * depth;
```

Comparing before and after, the usability gain with structured bindings alone is marginal, but it allows us to use descriptive names for the extents to improve readability.

The proposed feature is increasingly useful when structured bindings can introduce a pack, as proposed in [P1061R4] and shown below:

With P1061

```
std::mdspan<double, std::extents<I, Is...>, L, A> mdspan;
const auto& [...es] = mdspan.extents();
for (const auto [...is] : std::views::cartesian_product(std::views::iota(0, es)...))
    mdspan[is...] = 42.0;

const auto total = (es * ...);
```

In this example, we trade readability for generality. Destructuring the extents into a pack allows us to expand the extents again into a series of `iota` views, which we can turn into the index space for `mdspan` using a `cartesian_product`. Notice, that the implementation is also rank agnostic, and for `std::layout_right` (`std::mdspan`'s default) iterates contiguously through memory.

§ 2. Impact On the Standard

This is a pure library extension. Destructuring `std::extents` in the current specification [N4944] is ill-formed, because `std::extents` stores its runtime extents in a private non-static data member, which is inaccessible to structured bindings.

§ 3. Design Decisions

§ 3.1. Handling static extents

When destructuring `std::extents` we can deal with the compile-time/static extents in two ways:

- Option A: Demote the compile-time/static extents to runtime values.
- Option B: Retain the compile-time nature using e.g. `std::integral_constant`.

Option A is arguably simpler and may be less surprising. The structured bindings just represent what the `extents(i)` member function would return. Option B retains the compile-time nature of static extents at the cost of imposing a mix of integers and `std::integral_constants` upon users.

Regarding optimization potential, option A should rarely be a problem since structured bindings refer to a concrete instance of `std::extents` in scope, which is thus visible to the compiler. Using constant propagation, the compiler can likely determine the compile-time value that the structured bindings refer to. In the provided example implementation below, g++ 12.2 successfully unrolls the nested loops and transforms them into vector instructions.

It's worthwhile to point out that `std::integral_constant` has a non-explicit conversion operator to its `value_type`, so it will be demoted automatically to a runtime value where needed (e.g. in all examples above).

§ 3.2. Modification of extents

Modifications of the values stored inside a `std::extents` should not be allowed, since it is neither possible in case of a static extent nor does it follow the design of `std::extents::extent(rank_type) -> index_type`, which returns by value.

§ 4. Implementation Option A: demote static extents to runtime values

One possible implementation is to use the tuple interface and delegate to `std::extents::extent(rank_type)`:

```
namespace std {
template <size_t I, typename IndexType, size_t... Extents>
    constexpr IndexType get(const extents<IndexType, Extents...>& e) noexcept {
        return e.extent(I);
    }
}

template <typename IndexType, std::size_t... Extents>
struct std::tuple_size<std::extents<IndexType, Extents...>>
    : std::integral_constant<std::size_t, sizeof...(Extents)> {};

template <std::size_t I, typename IndexType, std::size_t... Extents>
struct std::tuple_element<I, std::extents<IndexType, Extents...>> {
    using type = IndexType;
};
```

An example of such an implementation using the Kokkos reference implementation of `std::mdspan` on Godbolt is provided here: https://godbolt.org/z/zo5Wb6TMG.

§ 5. Implementation Option B: retaining static extents

One possible implementation is to use the tuple interface and query the extents type whether a specific extent is static or not. Depending on this information, either the runtime extent via `std::extents::extent(I)` or a `std::integral_constant` of the appropriate index type and static extent is returned:

```
namespace std {
    template <size_t I, typename IndexType, size_t... Extents>
    constexpr auto get(const extents<IndexType, Extents...>& e) noexcept {
        if constexpr (extents<IndexType, Extents...>::static_extent(I) == dynamic_extent)
            return e.extent(I);
        else
            return integral_constant<IndexType,
```

```
                static_cast<IndexType>(
                    extents<IndexType, Extents...>::static_extent(I))>{};
    }
}

template <typename IndexType, std::size_t... Extents>
struct std::tuple_size<std::extents<IndexType, Extents...>>
    : std::integral_constant<std::size_t, sizeof...(Extents)> {};

template <std::size_t I, typename IndexType, std::size_t... Extents>
struct std::tuple_element<I, std::extents<IndexType, Extents...>> {
    using type = decltype(std::get<I>(std::extents<IndexType, Extents...>{}));
};
```

An example of such an implementation using the Kokkos reference implementation of `std::mdspan` on Godbolt is provided here: https://godbolt.org/z/841PeWM18.

6. Polls

The author would like to seek guidance on whether structured bindings for `std::extents` are perceived as useful and whether to continue with this proposal. And if yes, whether implementation Option A or Option B is preferred.

7. Wording

TODO

7.1. Feature-test macro

Add the following macro definition to 17.3.2 [version.syn], Header `<version>` synopsis, with the value selected by the editor to reflect the date of adoption of this paper:

```
#define __cpp_lib_extents_structured_bindings 20XXXXL // also in <mdspan>
```

8. Acknowledgements

I would like to thank Mark Hoemmen and Christan Trott for encouraging me to write this proposal, Mark Hoemmen for suggesting implementation B, and Michael Hava for reviewing.

References

Informative References

[N4944]
Thomas Köppe. *Working Draft, Standard for Programming Language C++*. 2023-03-19. URL: https://wg21.link/N4944

[P0009r18]
Christian Trott; et al. *MDSPAN*. 2022-07-13. URL: https://wg21.link/P0009r18

[P1061R4]

Barry Revzin; Jonathan Wakely. *Structured Bindings can introduce a Pack*. 2023-02-14. URL: https://wg21.link/P1061R4